

LAB ASSIGNMENT 20.5

NAME: R. PRANITHA REDDY

ID.NO:2303A52248

TASK 1: Password Storage Vulnerability Check

PROMPT:

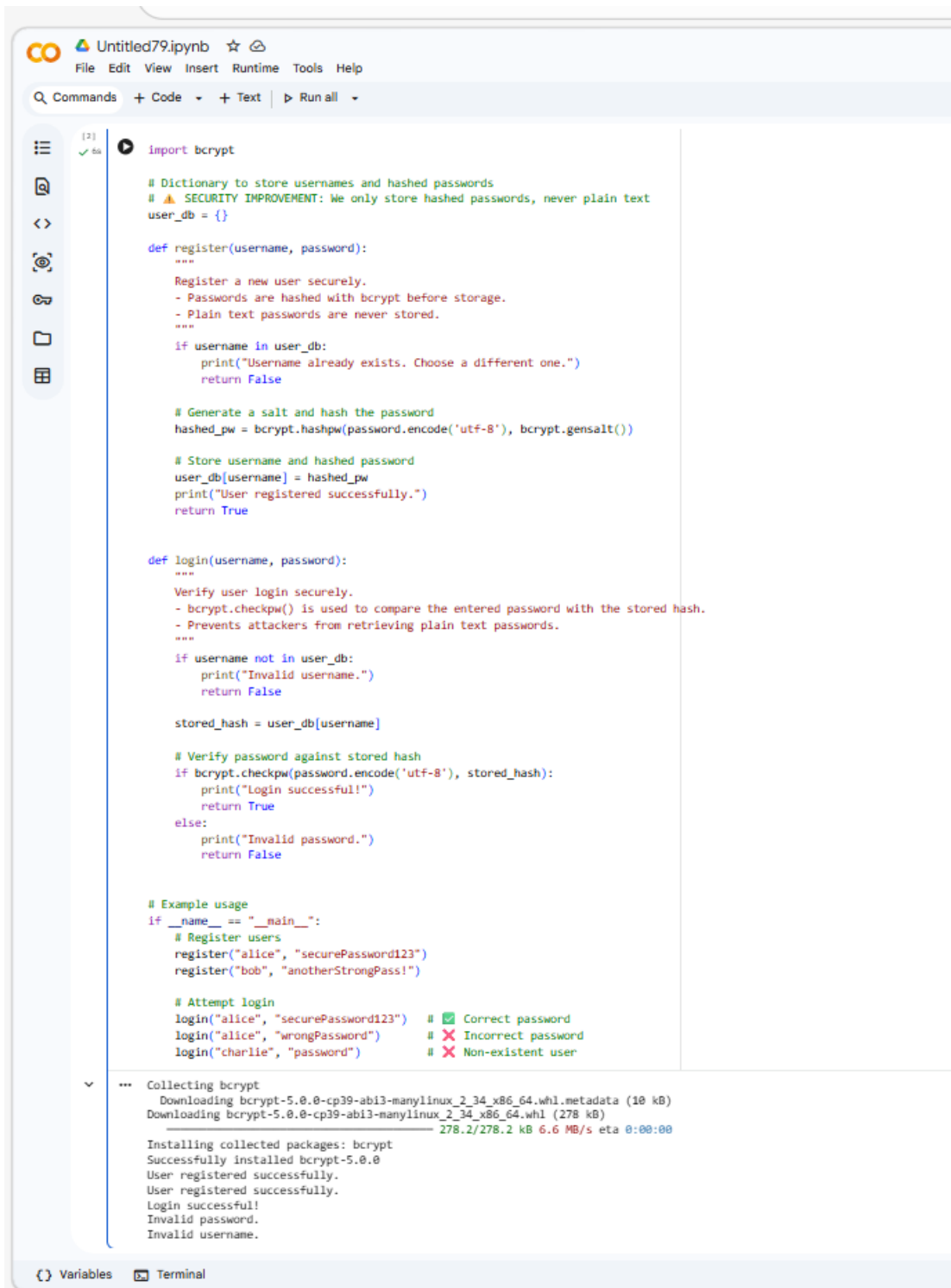
Rewrite the Python user registration system to make it secure:

Use bcrypt to hash passwords before storing Store usernames and hashed passwords in a dictionary Add a login function to verify passwords using bcrypt.checkpw() Do not store plain text passwords anywhere Add clear comments explaining security improvements.

EXPLANATION:

This task checks whether a signup system stores passwords insecurely in plain text, which is risky because anyone accessing the data can see user passwords. To fix this, passwords are converted into hashed values using tools like bcrypt or hashlib. Hashing ensures that even if the data is leaked, original passwords cannot be easily retrieved. This improves security and protects user credentials.

CODE:



The screenshot shows a Jupyter Notebook titled 'Untitled79.ipynb'. The interface includes a top menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The left sidebar contains icons for file management and execution. The main area displays Python code for a secure login system using bcrypt. The code includes functions for user registration and login, with comments explaining the security improvements. The output shows the execution of the code, including the installation of bcrypt and the results of the login attempts.

```
[2] ✓ 5s ▶ import bcrypt

# Dictionary to store usernames and hashed passwords
# ⚠ SECURITY IMPROVEMENT: We only store hashed passwords, never plain text
user_db = {}

def register(username, password):
    """
    Register a new user securely.
    - Passwords are hashed with bcrypt before storage.
    - Plain text passwords are never stored.
    """
    if username in user_db:
        print("Username already exists. Choose a different one.")
        return False

    # Generate a salt and hash the password
    hashed_pw = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

    # Store username and hashed password
    user_db[username] = hashed_pw
    print("User registered successfully.")
    return True

def login(username, password):
    """
    Verify user login securely.
    - bcrypt.checkpw() is used to compare the entered password with the stored hash.
    - Prevents attackers from retrieving plain text passwords.
    """
    if username not in user_db:
        print("Invalid username.")
        return False

    stored_hash = user_db[username]

    # Verify password against stored hash
    if bcrypt.checkpw(password.encode('utf-8'), stored_hash):
        print("Login successful!")
        return True
    else:
        print("Invalid password.")
        return False

# Example usage
if __name__ == "__main__":
    # Register users
    register("alice", "securePassword123")
    register("bob", "anotherStrongPass!")

    # Attempt login
    login("alice", "securePassword123") # ✅ Correct password
    login("alice", "wrongPassword")    # ❌ Incorrect password
    login("charlie", "password")       # ❌ Non-existent user
```

Collecting bcrypt
Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
278.2/278.2 kB 6.6 MB/s eta 0:00:00
Installing collected packages: bcrypt
Successfully installed bcrypt-5.0.0
User registered successfully.
User registered successfully.
Login successful!
Invalid password.
Invalid username.

Variables Terminal

TASK 2: Hardcoded Secrets Detection

PROMPT:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions: Ask AI to generate a script that connects to an external API. Identify if API keys/passwords are written directly in code.

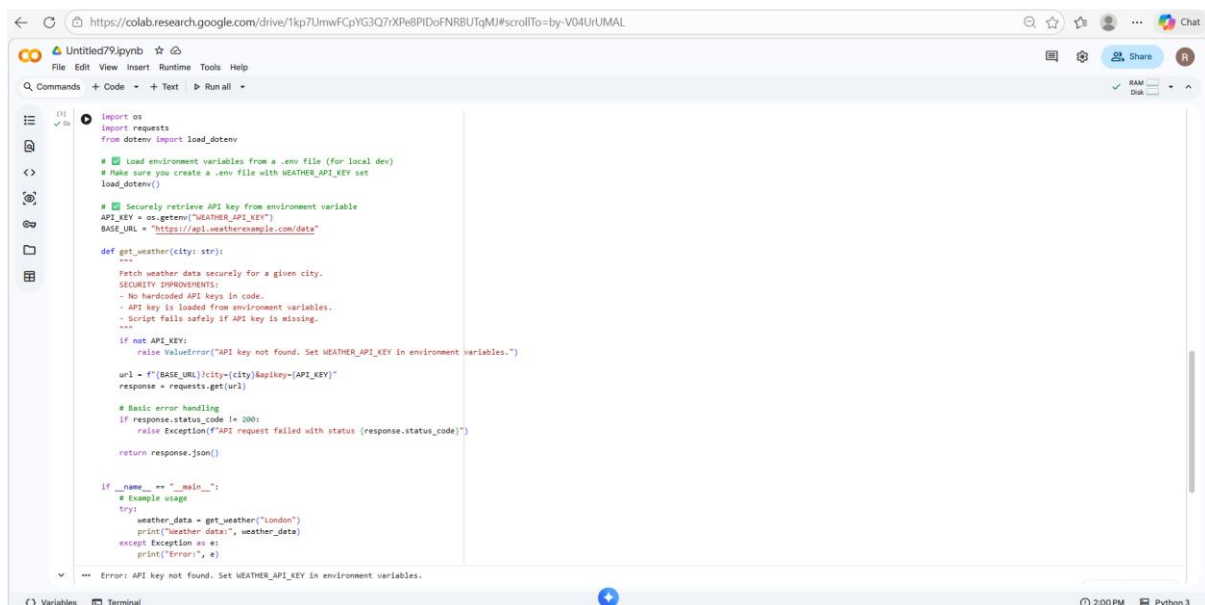
Refactor using environment variables or config files. Expected

Output: A secure script with secrets managed properly.

EXPLANATION:

Hardcoded secrets mean storing API keys or passwords directly in the source code, which is unsafe because anyone with access to the code can misuse them. This can lead to unauthorized access and data breaches. A secure approach is to store secrets in environment variables or configuration files instead of the code. This keeps sensitive information protected and separate from the application.

CODE:



```
import os
import requests
from dotenv import load_dotenv

# Load environment variables from a .env file (for local dev)
# Make sure you create a .env file with WEATHER_API_KEY set
load_dotenv()

# Securely retrieve API key from environment variable
API_KEY = os.getenv("WEATHER_API_KEY")
BASE_URL = "https://api.weatherexample.com/data"

def get_weather(city: str):
    """
    Fetch weather data securely for a given city.
    SECURITY IMPROVEMENTS:
    - No hardcoded API keys in code.
    - API key is loaded from environment variables.
    - Script fails safely if API key is missing.
    """
    if not API_KEY:
        raise ValueError("API key not found. Set WEATHER_API_KEY in environment variables.")

    url = f"{BASE_URL}?city={city}&apikey={API_KEY}"
    response = requests.get(url)

    # Basic error handling
    if response.status_code != 200:
        raise Exception(f"API request failed with status {response.status_code}")

    return response.json()

if __name__ == "__main__":
    # Example usage
    try:
        weather_data = get_weather("London")
        print("Weather data:", weather_data)
    except Exception as e:
        print("Error:", e)
```

Error: API key not found. Set WEATHER_API_KEY in environment variables.

TASK 3: File Upload Security Validation

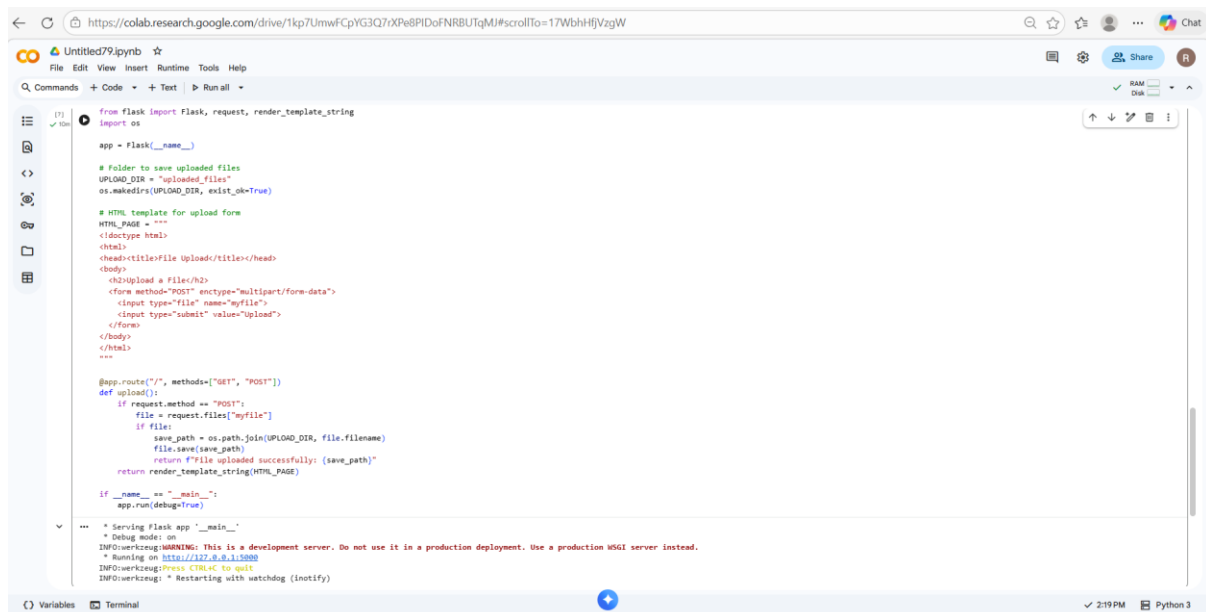
PROMPT:

Create a simple Flask application that: Allows users to upload a file, Saves the uploaded file to a folder on the server. Does not include any validation or restrictions. Keep the code simple and beginner-friendly

EXPLANATION:

Unrestricted file uploads are dangerous because attackers can upload malicious files like scripts or executables to the server. This can lead to server compromise or data breaches. To prevent this, files should be validated by type and size, and filenames should be sanitized using secure methods. These measures ensure only safe files are accepted and stored securely.

CODE:



```
from flask import Flask, request, render_template_string
import os

app = Flask(__name__)

# Folder to save uploaded files
UPLOAD_DIR = "uploaded_files"
os.makedirs(UPLOAD_DIR, exist_ok=True)

# HTML template for upload form
HTML_PAGE = """
<doctype html>
<html>
<head><title>File Upload</title></head>
<body>
<h2>Upload a File</h2>
<form method="POST" enctype="multipart/form-data">
  <input type="file" name="myfile">
  <input type="submit" value="Upload">
</form>
</body>
</html>
"""

@app.route("/", methods=["GET", "POST"])
def upload():
    if request.method == "POST":
        file = request.files["myfile"]
        if file:
            save_path = os.path.join(UPLOAD_DIR, file.filename)
            file.save(save_path)
            return f"File uploaded successfully: {save_path}"
        return render_template_string(HTML_PAGE)

    if __name__ == "__main__":
        app.run(debug=True)
```

... " Serving Flask app "main..."
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on <http://127.0.0.1:5000>
INFO:werkzeug:Press CTRL-C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)

TASK 4: Command Injection Prevention

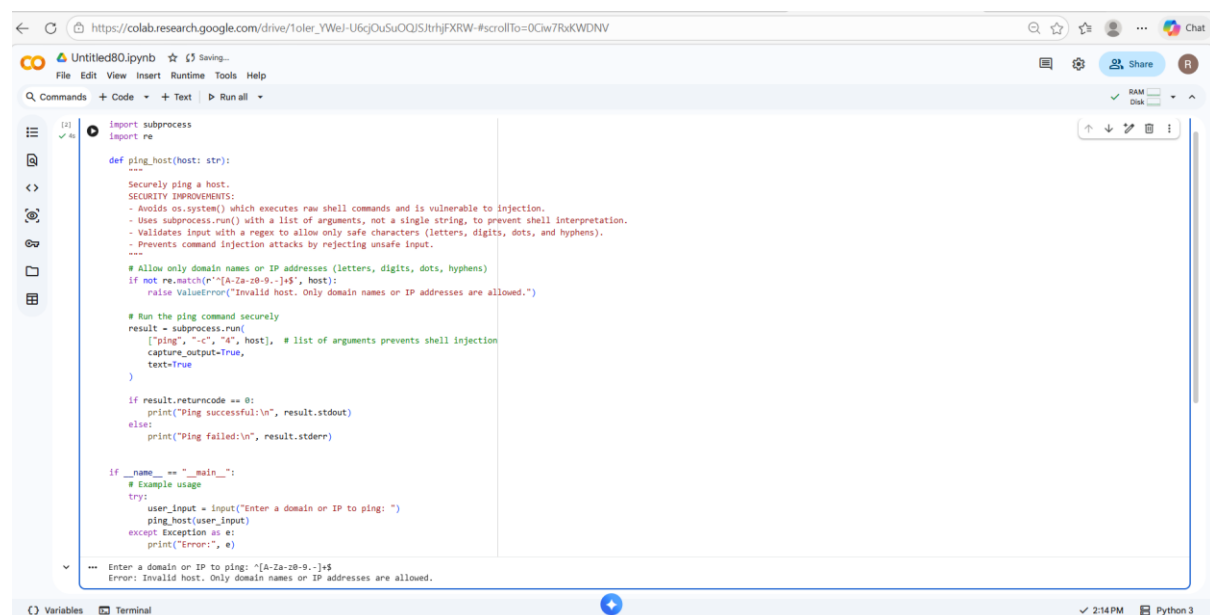
PROMPT:

Rewrite the Python script to make it secure: Avoid using `os.system()` with user input. Use `subprocess.run()` with a list of arguments instead of a single string, Validate and sanitize user input (allow only safe characters like domain names or IPs). Prevent command injection attacks, Add comments explaining the security improvements

EXPLANATION:

Command injection occurs when user input is directly used in system commands, allowing attackers to execute malicious commands on the server. Using functions like `os.system()` with raw input makes the system vulnerable. To prevent this, use `subprocess.run()` with argument lists and avoid shell execution. Additionally, validating user input ensures only safe values are processed.

CODE:



```
import subprocess
import re

def ping_host(host: str):
    """
    Securely ping a host.
    SECURITY IMPROVEMENTS:
    - Avoids os.system() which executes raw shell commands and is vulnerable to injection.
    - Uses subprocess.run() with a list of arguments, not a single string, to prevent shell interpretation.
    - Validates input with a regex to allow only safe characters (letters, digits, dots, and hyphens).
    - Prevents command injection attacks by rejecting unsafe input.
    """
    # Allow only domain names or IP addresses (letters, digits, dots, hyphens)
    if not re.match(r"^[A-Za-z0-9-]*$", host):
        raise ValueError("Invalid host. Only domain names or IP addresses are allowed.")

    # Run the ping command securely
    result = subprocess.run(
        ["ping", "-c", "4", host], # list of arguments prevents shell injection
        capture_output=True,
        text=True
    )

    if result.returncode == 0:
        print("Ping successful:\n", result.stdout)
    else:
        print("Ping failed:\n", result.stderr)

if __name__ == "__main__":
    # Example usage
    try:
        user_input = input("Enter a domain or IP to ping: ")
        ping_host(user_input)
    except Exception as e:
        print("Error:", e)
```

Enter a domain or IP to ping: ^[A-Za-z0-9-]*\$
Error: Invalid host. Only domain names or IP addresses are allowed.

TASK 5: Secure Input Handling in Web Forms

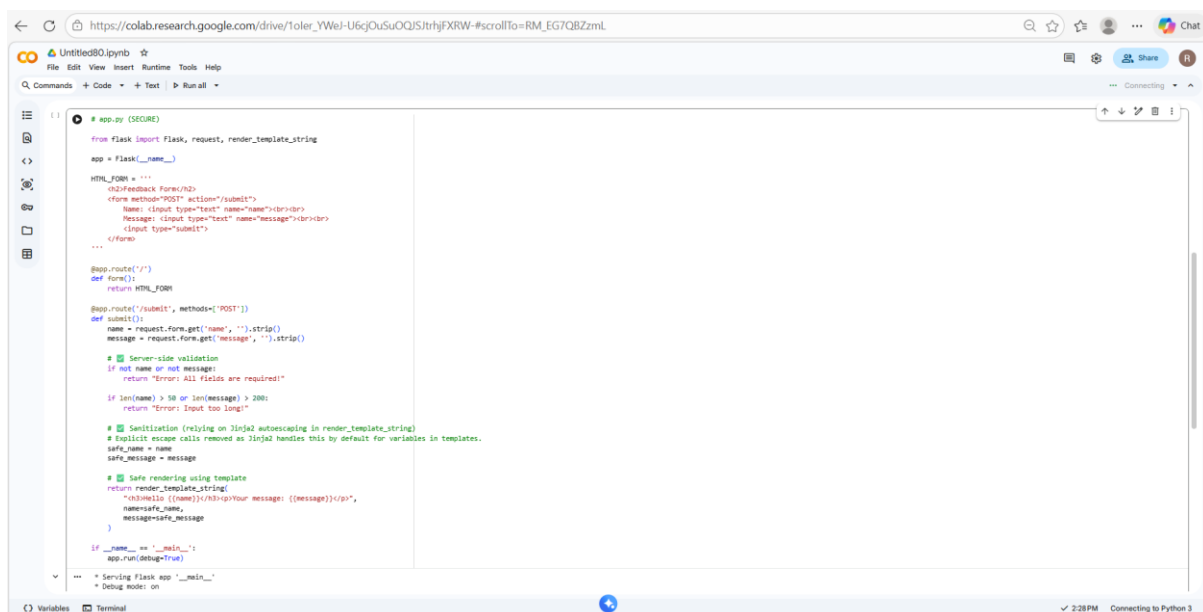
PROMPT:

Rewrite the Flask web form application to make it secure: Add server-side validation for all inputs (e.g., non-empty, length limits). Sanitize user input before displaying it. Use Jinja2 autoescaping or escape functions to prevent XSS. Avoid directly rendering raw user input. Add proper error handling and messages. Add comments explaining security improvements

EXPLANATION:

Improper input handling in web forms can allow attackers to inject malicious scripts, leading to Cross-Site Scripting (XSS) attacks. This happens when user input is displayed without validation or sanitization. Secure applications validate inputs on the server side and escape or sanitize data before rendering. This prevents harmful code from executing in the browser.

CODE:

The image shows a Google Colab notebook interface with a code editor and a terminal. The code is for a Flask web application named 'app.py' with a security focus. It includes imports for Flask, request, and render_template_string. A route '/' returns an HTML form with two input fields: 'name' and 'message'. A POST route '/submit' processes the form data. It includes server-side validation: checking if both fields are present and if the message length is under 200 characters. It then sanitizes the inputs using Jinja2's autoescaping (commented out) and escape functions (commented out). The final output is rendered using Jinja2's safe rendering (commented out). The terminal shows the application running on port 5000.

```
1 # app.py (SECURE)
2
3 from flask import Flask, request, render_template_string
4
5 app = Flask(__name__)
6
7 HTML_FORM = '''
8 <html>
9 <head>
10 <title>Feedback Form</title>
11 </head>
12 <body>
13 <form method="POST" action="/submit">
14   Name: <input type="text" name="name"><br/>
15   Message: <input type="text" name="message"><br/>
16   <input type="submit" value="Submit" />
17 </form>
18 </body>
19 </html>
20 '''
21
22 @app.route("/")
23 def form():
24     return HTML_FORM
25
26 @app.route("/submit", methods=['POST'])
27 def submit():
28     name = request.form.get('name', '').strip()
29     message = request.form.get('message', '').strip()
30
31     # Server-side validation
32     if not name or not message:
33         return "Error: All fields are required!"
34
35     if len(name) > 50 or len(message) > 200:
36         return "Error: Input too long!"
37
38     # Sanitization (relying on Jinja2 autoescaping in render_template_string)
39     # Explicit escape calls removed as Jinja2 handles this by default for variables in templates.
40     safe_name = name
41     safe_message = message
42
43     # Safe rendering using template
44     return render_template_string(
45         """<h3>Hello {{name}}!</h3><p>Your message: {{message}}</p>""",
46         name=safe_name,
47         message=safe_message
48     )
49
50 if __name__ == '__main__':
51     app.run(debug=True)
```